

VARREDURA DE SCANNERS DAST EM UMA APLICAÇÃO DELIBERADAMENTE

VULNERÁVEL

UMA ANÁLISE COMPARATIVA

Acadêmico: Eric Coca Brisola

Orientador: Prof. Dr. Mário Ezequiel Augusto

Sistemas de informação
2026/1



Sumário

- Introdução;
- Problemática;
- Objetivos;
- Fundamentação Teórica;
- Metodologia;
- Resultados;
- Discussão;
- Limitações;
- Trabalhos futuros;
- Conclusão;

Introdução

- Segurança web tornou-se requisito obrigatório — falhas expõem dados e geram prejuízos financeiros e reputacionais (Sampaio, 2021);
- **49%** das aplicações contêm vulnerabilidades graves, **13%** expostas a falhas críticas (Rosa et al., 2024);
- Front-end é o principal alvo — falhas como SQLi, XSS e Quebra de Controle de Acesso lideram incidentes (Tkachenko et al., 2024; Somi, 2024);

Introdução

- Scanners DAST automatizam a detecção simulando ataques externos sem acesso ao código-fonte;
- Porém: ampla variedade de opções dificulta a escolha da ferramenta mais adequada (Somi, 2024);
- Confiar em uma única ferramenta isolada é insuficiente (Rosa et al., 2024);

Problemática

- Dificuldade de escolher um scanner DAST de código aberto;
- Literatura foca muito em eficácia de detecção;
- Consumo de recursos, qualidade de relatório e consistência são pouco avaliados;

Objetivos

- Realizar análise empírica e comparativa entre scanners DAST de código aberto;
- OWASP ZAP 2.17.0 e Wapiti 3.2.10;
- Avaliar desempenho por meio de 8 métricas separadas em 3 categorias:
 - Eficácia de detecção;
 - Eficiência operacional;
 - Qualidade do relatório;

Fundamentação teórica

O que é DAST?

Dynamic Application Security Testing (DAST)

- Testa a aplicação **em execução**, sem acesso ao código-fonte;
- Blackbox: acesso somente externo (sem credenciais)
- Graybox: análise externa + conhecimento parcial (ex.: autenticação)

Fundamentação teórica

O que é DAST?

Os 3 módulos de um scanner DAST:

- **Crawler** → mapeia rotas e pontos de entrada (GET, POST, formulários);
- **Fuzzer** → injeta payloads maliciosos nos endpoints descobertos;
- **Reporter** → gera relatório com alertas, severidade e evidências;

Fundamentação teórica

As ferramentas avaliadas

OWASP ZAP

Zed Attack Proxy — ZAP Dev Team

- Varredura ativa e passiva;
- Interface gráfica + linha de comando;
- Suporte Blackbox e Graybox;
- Open source, amplamente adotado;

Versão: 2.17.0

Fundamentação teórica

As ferramentas avaliadas

Wapiti

Nicolas Surribas — linha de comando

- Focado em varreduras Blackbox;
- Escrito em Python, leve e rápido;
- Rastreia formulários e parâmetros GET/POST;
- Open source, manutenção ativa;

Versão: 3.2.10

Fundamentação teórica

Ambiente de teste: OWASP Juice Shop

- Aplicação deliberadamente vulnerável desenvolvida pelo projeto OWASP;
- Stack: Node.js + Express (backend) + Angular (frontend) — **SPA**;
- **+100 desafios** de segurança em 16 categorias;
- Cobre todas as categorias do **OWASP Top 10 2021**;
- Referência amplamente adotada para avaliar scanners na literatura;
- *Desafio: SPA (Angular) renderiza via JavaScript → crawler convencional cobre apenas endpoints públicos;*

Metodologia

- **Protocolo de execução:**

- Cada scanner foi executado 3 vezes (runs) em modo Blackbox, sem autenticação;
- Monitor de recursos (monitor.sh) coletou RAM e CPU durante a varredura;

- **Ambiente de execução:**

- Windows 11 + WSL2 + Docker Desktop
- Notebook Acer Nitro 5 AN515-54
- Intel i5-9300H - 16 GB RAM
- Juice Shop: container Docker
- Scanner: docker run (processo único)
- monitor.sh: coleta RAM e CPU em paralelo

Metodologia

Métricas - Eficácia de Detecção

- **M1 — Taxa de verdadeiro Positivo por categoria Juice Shop (TVP):**
 - Taxa de vulnerabilidades reais encontradas e as separa pelas categoria;
- **M2 — Taxa de Falsos Positivos (TFP):**
 - Taxa de alertas que não são vulnerabilidades reais, mesmo que não esteja mapeada no Juice Shop;
- **M3 — Cobertura de Endpoints:**
 - Proporção de rotas HTTP do Juice Shop visitadas pelos scanners em comparação ao total de rotas HTTP;

Metodologia

Métricas - Eficácia de Detecção – Fórmulas

- M1: $TVP = \frac{VD}{TVC}$
- M2: $TFP = \frac{FP}{(VP + FP)}$
- M3: $\frac{EP}{TE}$

VD: vulnerabilidades detectadas, TVC: total de vulnerabilidades da categoria no gabarito, FP: falso positivo, VP: verdadeiro positivo, EP: endpoints visitados, TE: total de endpoints do gabarito

■ Metodologia

Métricas - Eficiência Operacional

- **M4 — Tempo total de varredura:**
 - Mede tempo do inicial ao final da execução em segundos;
- **M5 — Consistência entre execuções:**
 - Avalia se os resultados se mantêm iguais em várias execuções;
- **M6a — Pico de RAM**
 - Mede maior valor de memória RAM registrado durante a execução;
- **M6b — Média CPU**
 - Mede o impacto da varredura sobre o processador durante a execução;

■ Metodologia

Métricas - Qualidade do Relatório

- **M7 — Completude da evidência:**

- Avalia a qualidade dos dados do relatório dos scanners, usando 5 critérios:

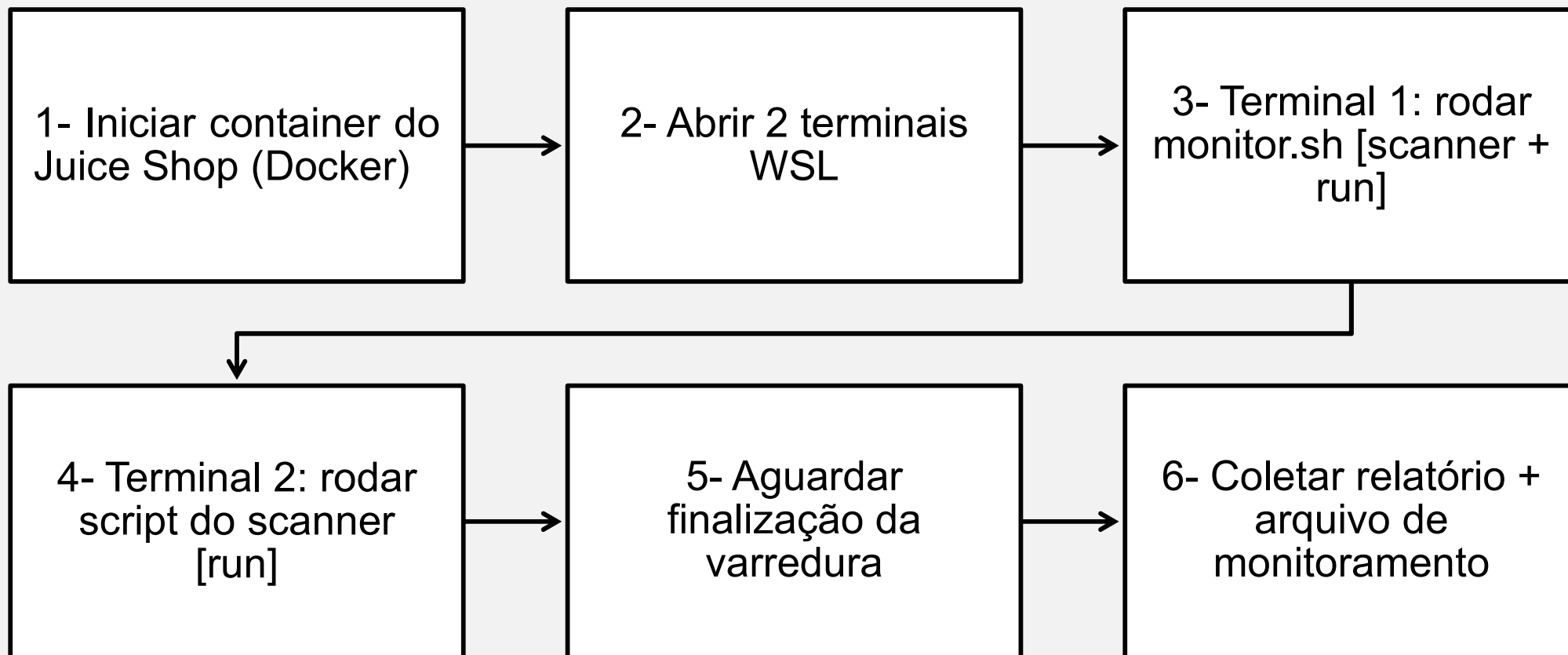
- Presença do payload de ataque;
 - Requisição HTTP;
 - Resposta HTTP;
 - Referência a um CWE (catálogo de vulnerabilidades);
 - Classificação da severidade;

- **M8 — Precisão da severidade:**

- Avalia a severidade fornecida pelos scanners nos relatórios com base no OWASP Top 10 (2021);

Metodologia

Fluxo de Execução

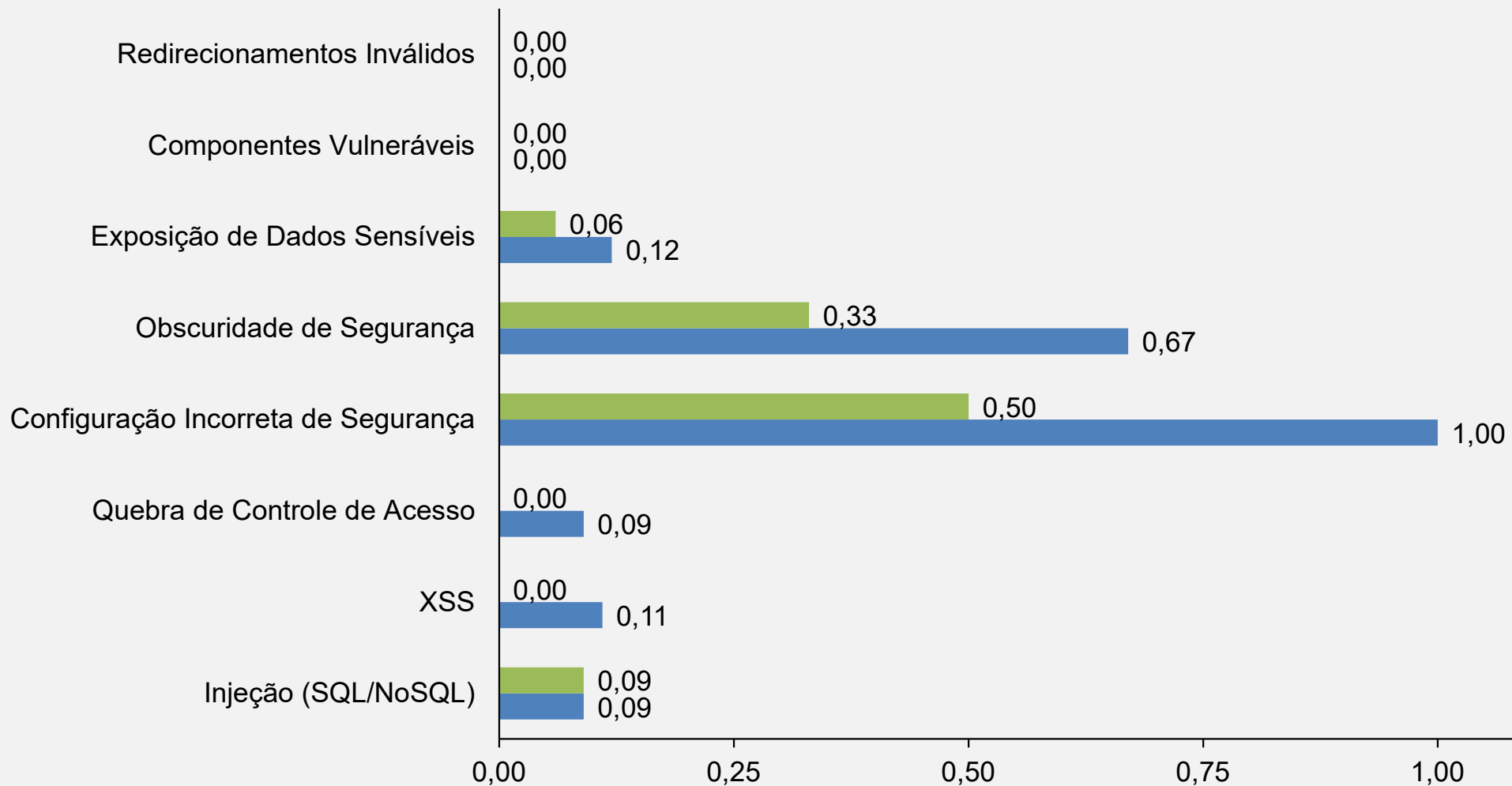


Resultados

Métricas - Eficácia de Detecção

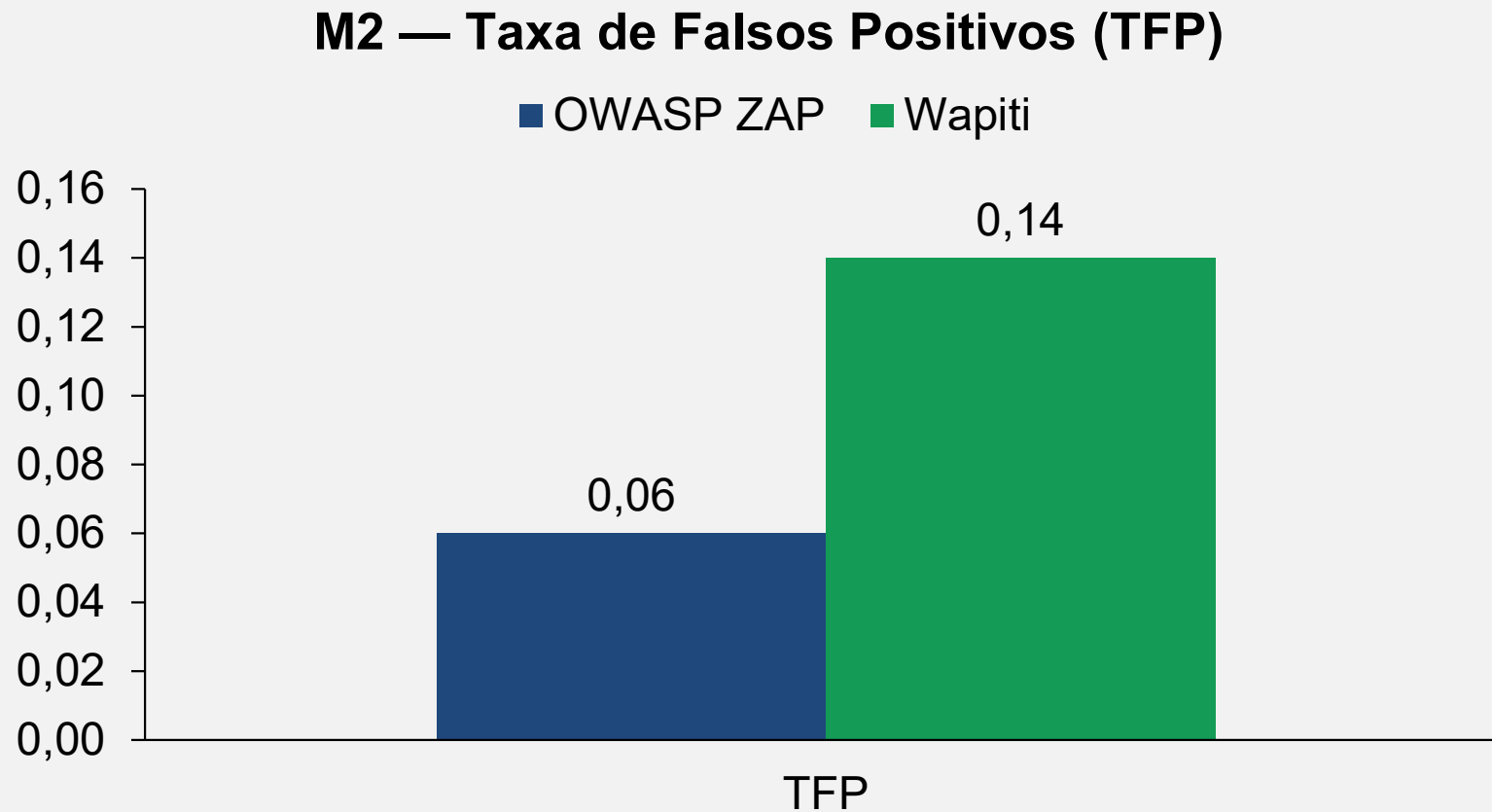
M1 — TVP por Categoria

■ Wapiti ■ OWASP ZAP



■ Resultados

Métricas - Eficácia de Detecção



Resultados

Métricas - Eficácia de Detecção

M3 — Cobertura

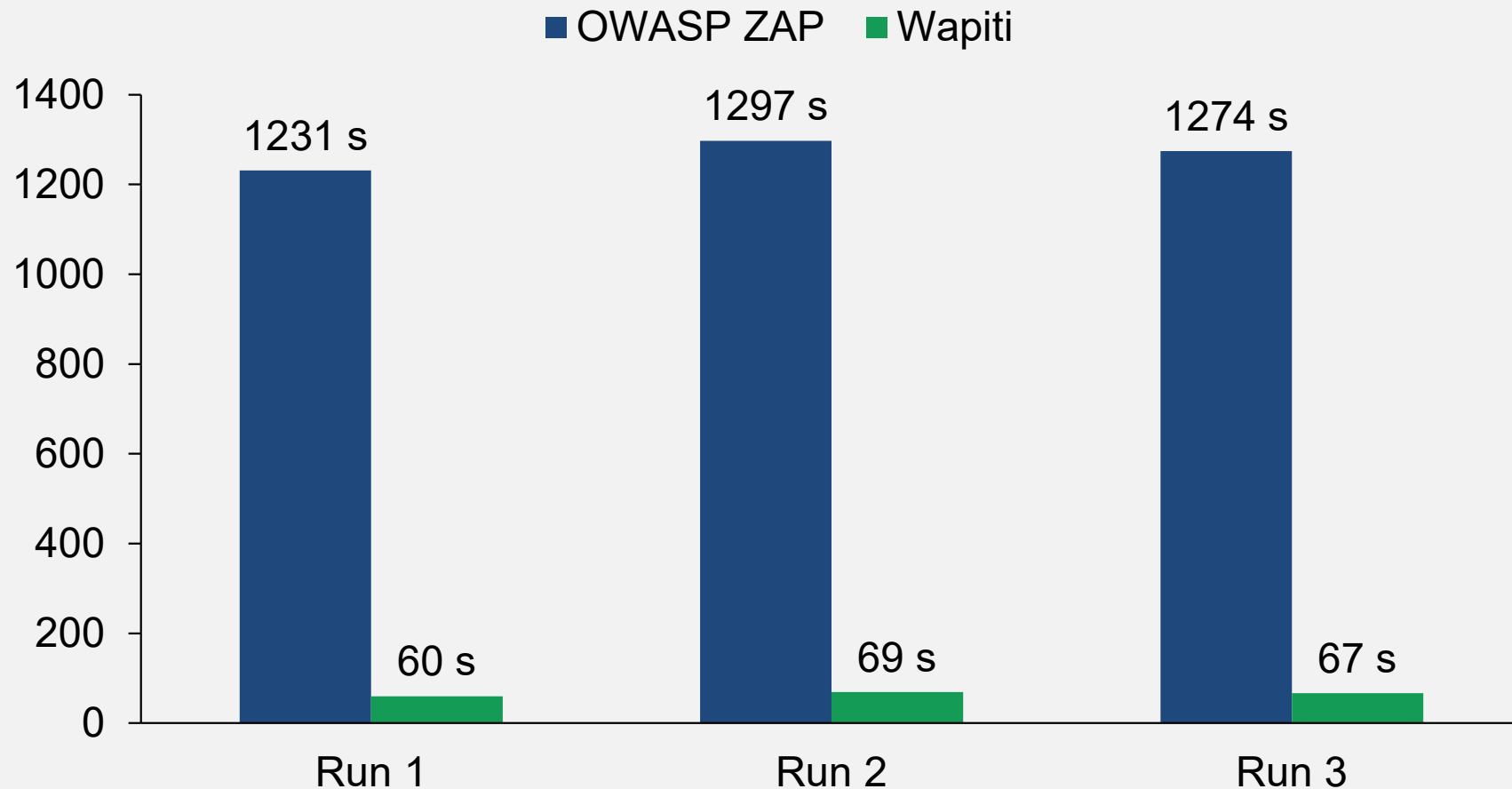
- ZAP: 11/84 (**13%**)
- Wapiti: 6/84 (**7%**)

Baixa cobertura esperada em varreduras sem autenticação

Resultados

Métricas - Eficiência Operacional

M4 — Tempo Total de Varredura por Execução (segundos)



Resultados

M5 — Consistência Entre Execuções

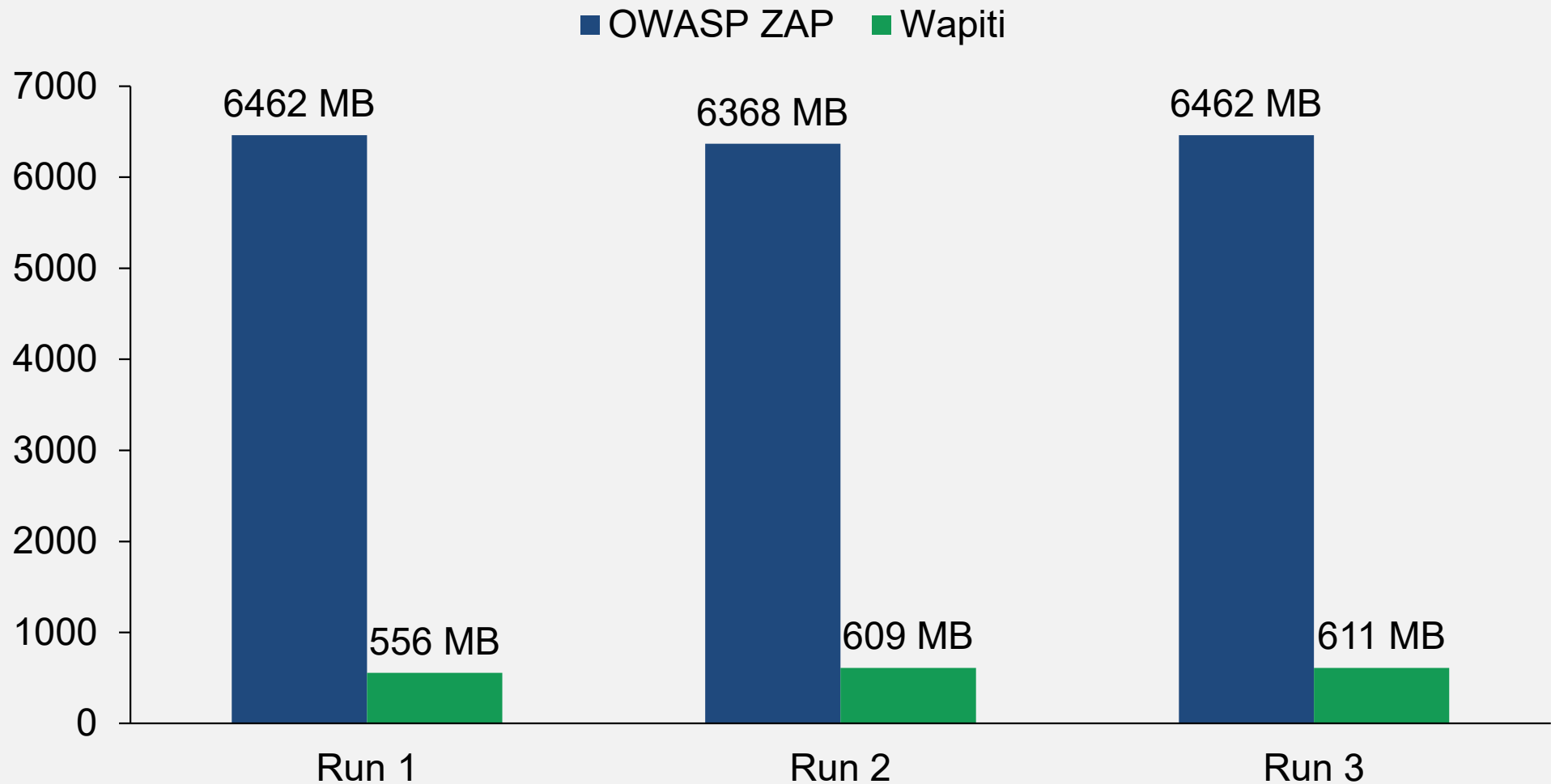
Ferramenta	Run 1	Run 2	Run 3	Variação	Consistência
ZAP — Categorias detectadas	16	16	16	0	100%
ZAP — URLs rastreadas	207	208	207	±1	99,5%
Wapiti — Categorias detectadas	6	6	6	0	100%
Wapiti — Páginas rastreadas	42	44	44	±2	95,5%

Consistência altíssima nas duas ferramentas

Obs: essas categorias encontradas são dos próprios scanners;

Resultados

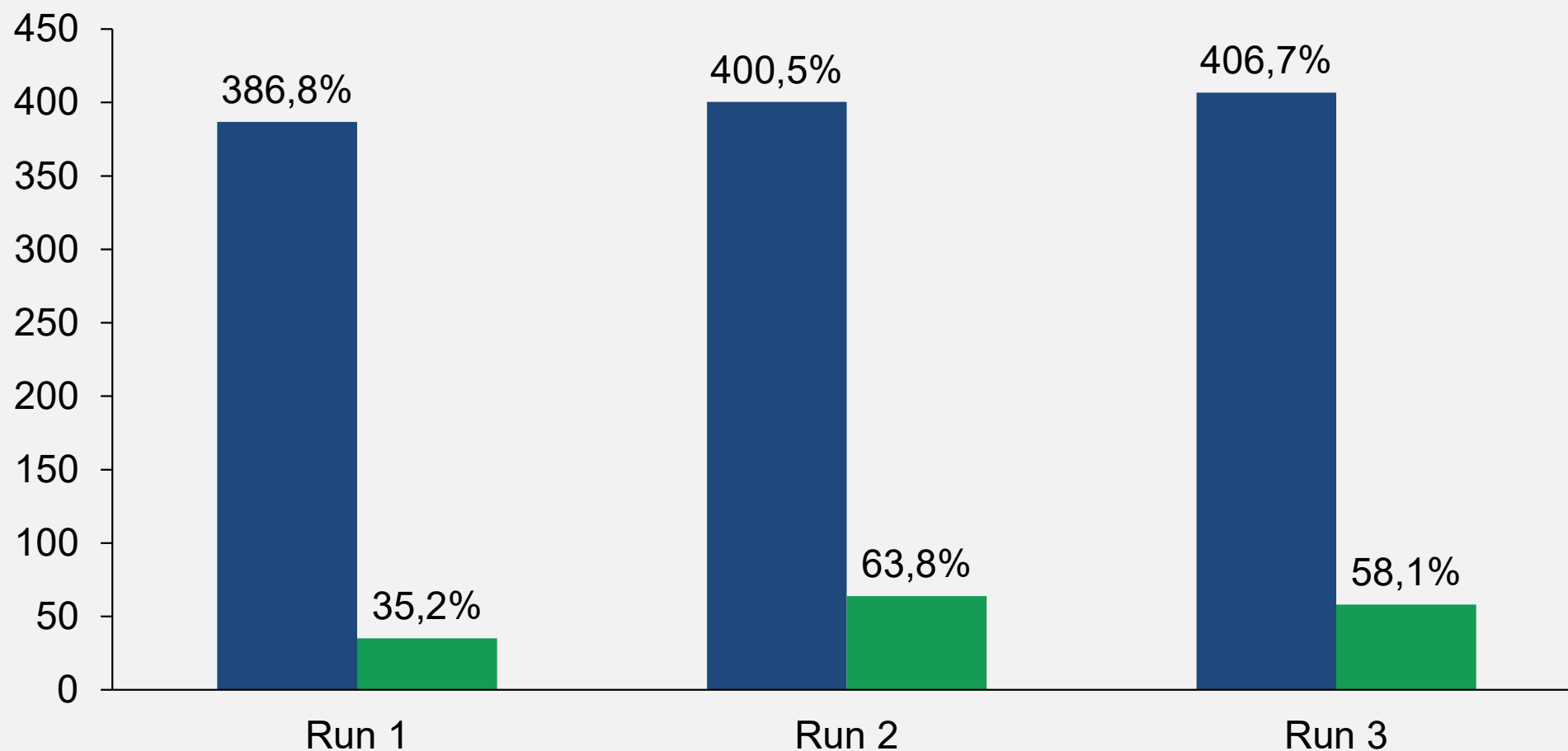
M6a — Pico de Consumo de RAM por Execução (MB)



Resultados

M6b — Média de Uso de CPU por Execução (%)

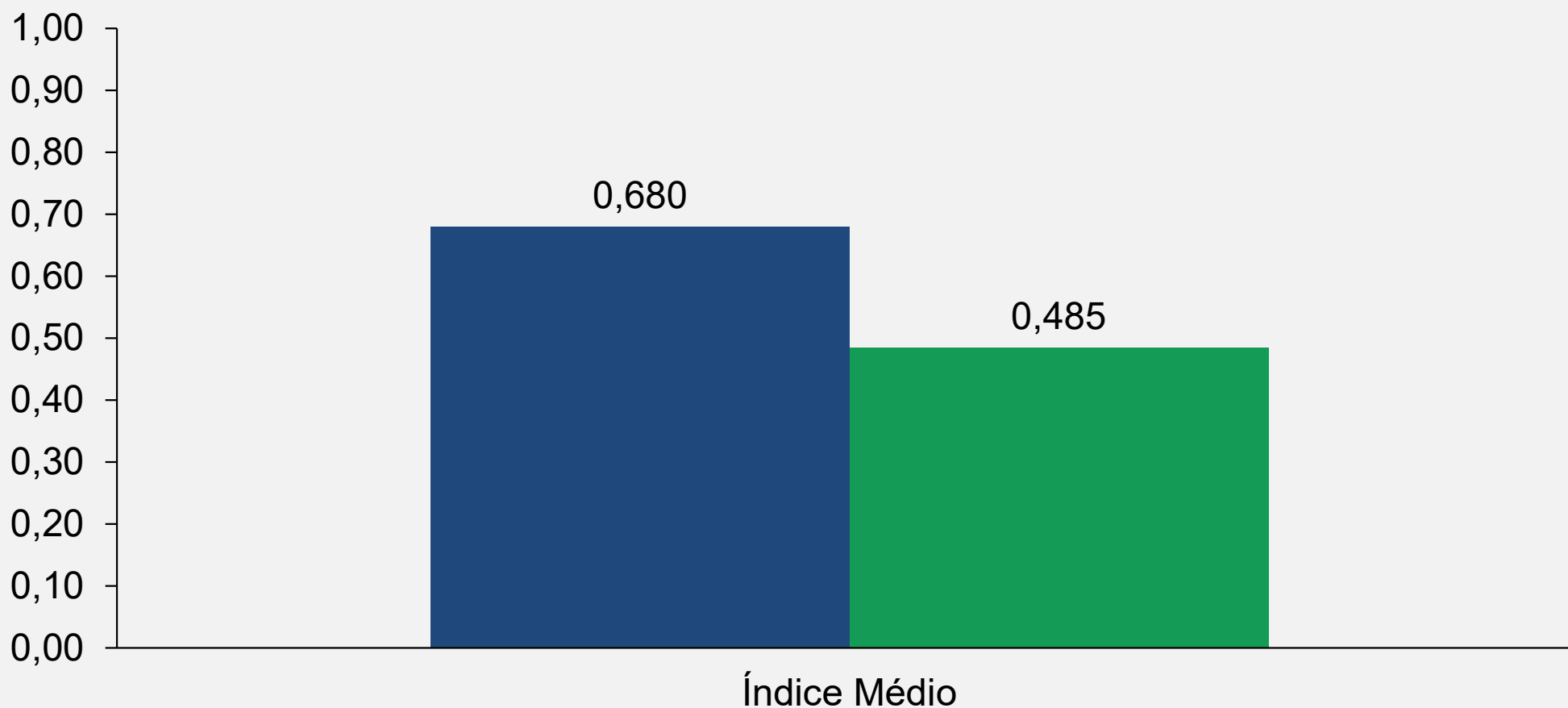
■ OWASP ZAP ■ Wapiti



Resultados

M7 — Completude de Evidência (Índice Médio, 0–1)

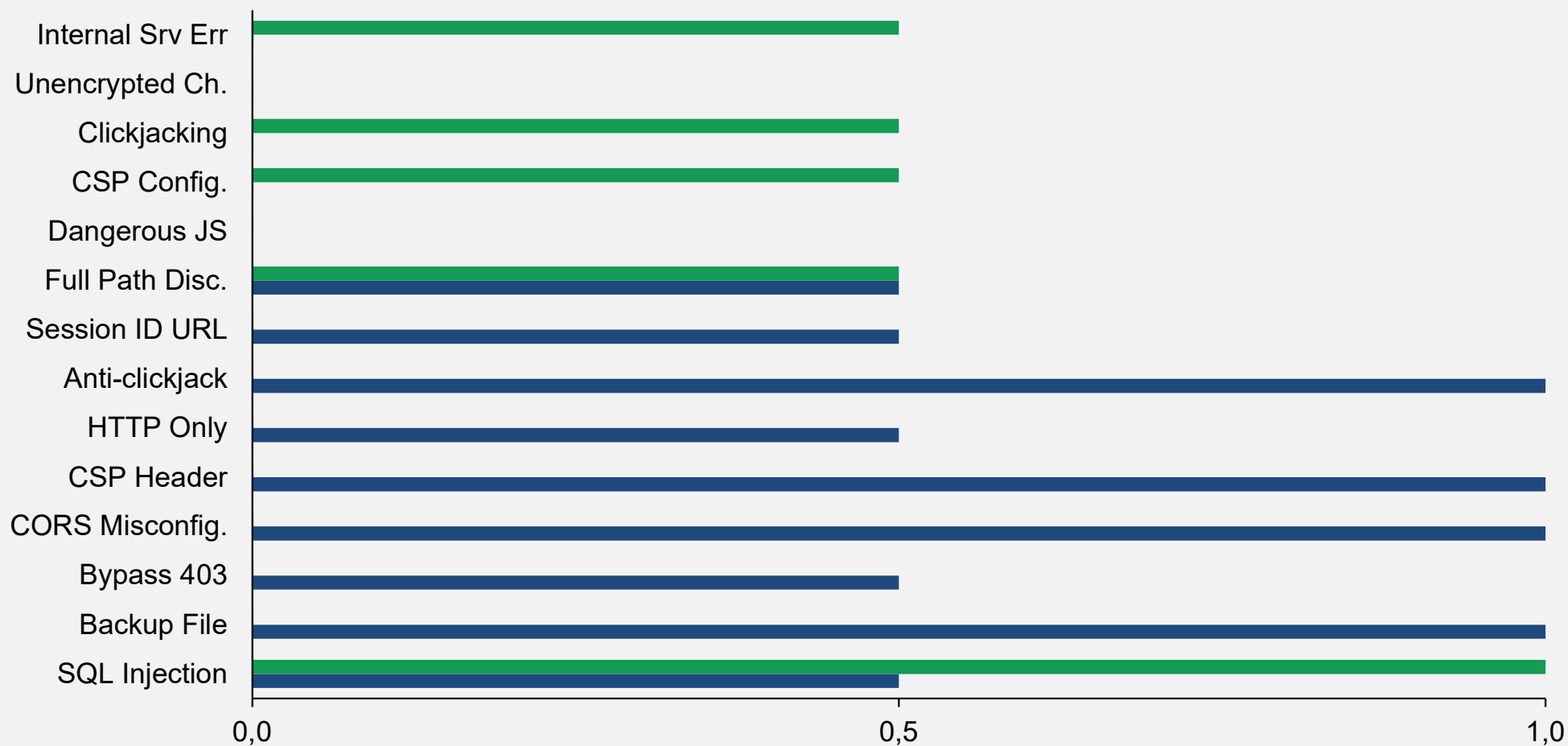
■ OWASP ZAP ■ Wapiti



Resultados

M8 — Precisão de Severidade por Alerta (0,0 / 0,5 / 1,0)

■ Wapiti ■ OWASP ZAP



Síntese Comparativa (M1–M8)

Métrica	ZAP	Wapiti
M1 — TVP (Configuração Incorreta de Segurança)	1,00	0,50
M2 — Taxa Falsos Positivos	0,06	0,14
M3 — Cobertura de Endpoints	13% (11/84)	7% (6/84)
M4 — Tempo médio	1.267s (~21 min)	65s (~1 min)
M5 — Consistência	99,5% – 100%	95,5% - 100%
M6a — Pico RAM	~6.430 MB	~592 MB
M6b — Média CPU	398% (4 núcleos)	52% (<1 núcleo)
M7 — Completude evidência	0,68	0,485
M8 — Precisão severidade	0,65	0,50

Discussão

Koman & Janiszewski (2025) — SCAnME

- Mesmo ambiente de teste (Juice Shop) e scanners;
- Configuração Incorreta de Segurança foi a mais detectada em ambas as ferramentas:
 - Não precisa de autenticação;
 - Detectável via cabeçalhos HTTP ausentes ou mal configurados;
- Wapiti sem crawler para JS tem menos alcance;
 - Usado Playwright + Firefox;
- ZAP com crawler integrado nativamente;

Discussão

Koman & Janiszewski (2025) — SCAnME

- Falsos positivos:
 - ZAP: 0 FP vs este trabalho: 1 FP;
 - Wapiti: 0 FP vs este trabalho 1FP ;
- Tempo da varredura (média em segundos):
 - ZAP: 464 vs este trabalho 1.267;
- SCAnMe:
 - Rodava direto no Kali Linux;
- Este trabalho:
 - Docker + WSL;
- + Tempo de processamento entre as camadas;

Discussão

Rosa et al. (2024) — 7 scanners no Juice Shop

- Recomenda uso de múltiplas ferramentas para maior cobertura;
- ZAP: melhor scanner dentre os 7;
- Nenhuma ferramenta cobre todas as categorias isolada;
- Nesse trabalho:
 - XSS e Quebra de Controle de Acesso: Apenas ZAP;
 - *Componentes Vulneráveis*: Nenhum;
 - Redirecionamentos Não Validados: Nenhum;

Limitações

- Varreduras sem autenticação — ~60 dos 84 endpoints bloqueados;
- Ambiente WSL + Docker no Windows gera overhead de virtualização;
- Apenas uma plataforma vulnerável avaliada (Juice Shop);

Trabalhos Futuros

- Varreduras autenticadas (Graybox) — avaliar endpoints protegidos por sessão;
- Outras plataformas: OWASP WebGoat, DVWA, BWAPP
- Incluir mais ferramentas DAST (Nikto, Burp Suite Community);
- Ambiente Linux nativo (sem WSL) para comparação de desempenho;

Conclusão

OWASP ZAP

Profundidade > Velocidade

- ✓ Mais verdadeiros positivos (M1)
- ✓ Menor taxa de falsos positivos (0,06)
- ✓ Relatório mais completo (0,68)
- ✗ 19,5× mais lento (1.267s)
- ✗ 10,9× mais RAM (6.462 MB)

Ideal: CI/CD com recursos e cobertura ampla

Wapiti

Velocidade > Profundidade

- ✓ Extremamente rápido (65s)
- ✓ Leve em RAM (591 MB)
- ✓ Alta consistência entre runs
- ✗ Menor cobertura (M3: 7%)
- ✗ Relatório menos detalhado (0,485)

Ideal: triagens rápidas / ambientes com recursos limitados

Referências

AL ANHAR, Azwar; SURYANTO, Yohan. Evaluation of web application vulnerability scanner for modern web application. In: **2021 International conference on artificial intelligence and computer science technology (ICAICST)**. IEEE, 2021. p. 200-204.

CURPHEY, M.; ARAWO, R. Web application security assessment tools. IEEE Security & Privacy, v. 4, n. 4, p. 32–41, 2006. DOI: 10.1109/MSP.2006.108.

DORA, Jean Rosemond; NEMOGA, Karol. Ontology for cross-site-scripting (xss) attack in cybersecurity. **Journal of Cybersecurity and Privacy**, v. 1, n. 2, p. 319-339, 2021.

KOMAN, Jakub; JANISZEWSKI, Marek. SCAnME-scanner comparative analysis and metrics for evaluation: J. Koman, M. Janiszewski. **International Journal of Information Security**, v. 24, n. 3, p. 147, 2025.

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **OWASP Top 10: 2021**. [S. l.], 2021. Disponível em: <https://owasp.org/Top10/>. Acesso em: 24 maio 2026.

ROSA, Ricardo et al. Análise empírica e comparativa de ferramentas de varredura de vulnerabilidades em aplicações Web usando OWASP BWA e Juice Shop. In: **Escola Regional de Redes de Computadores (ERRC)**. SBC, 2024. p. 183-188.

Referências

SAMPAIO, Felipe Ferreira. **Uma análise prática das principais vulnerabilidades em aplicações web baseado no top 10 OWASP**. 2021. Trabalho de Conclusão de Curso (Bacharelado) — Universidade Federal do Paraná, Paraná, 2021.

SOMI, V. A comparative analysis and benchmarking of dynamic application security testing (DAST) tools. **Journal of Engineering and Applied Sciences Technology**, v. 1, n. 6, p. 1-6, 2024.

STEINHAUSER, Antonín; TŮMA, Petr. Database traffic interception for graybox detection of stored and context-sensitive XSS. **Digital Threats: Research and Practice**, v. 1, n. 3, p. 1-23, 2020.

TKACHENKO, Oleksandr; GONCHAROV, Vadim; JATKIEWICZ, Przemysław. Enhancing Front-End Security: Protecting User Data and Privacy in Web Applications. **Computer Animation and Virtual Worlds**, v. 35, n. 6, p. e70003, 2024.



Obrigado!

Eric Coca Brisola

ericcoca13@gmail.com

Orientador: Prof. Dr. Mário Ezequiel Augusto
UDESC — Bacharelado em Sistemas de Informação

Perguntas?